2026

F1 Fan Hub Midterm Project

PROFEESOR: DR. LEONARDOS MAGEIROS
KONSTANTINOS ZERVAKOS
COURSE: ITC-4214A1
ID:236412
DEADLINE: 4 MARCH, 2026 BY MIDNIGHT 11:59

THE AMERICAN COLLGE OF GREECE

F1 Fan Hub – Web Application Report

Table of Contents

F1 Fan Hub – Web Application Report 1
1. Description of the Task Allocation System 1
2. Detailed Documentation of Coding Decisions 2
2.1 Tasks Page – Technical Decisions 2
2.2 Latest Activity Section 2
2.3 Personal (About) Page 2
2.4 API Integration Decision 2
2.5 Accessibility Considerations 3
3. GitHub Repository 3
4. Lighthouse Audit Results 3
..... 5
5. Additional Components Implemented 5
6. AI Usage 6
7. Reflections 7

1. Description of the Task Allocation System

The F1 Fan Hub task allocation system is a client-side task management solution designed for Formula 1 fans to organize race-related activities such as watch parties, travel plans, reminders, and event preparation.

Each task is stored as an object with the following properties:
- Unique ID (generated dynamically)
- Task Name
- Description
- Due Date
- Priority (High, Medium, Low)
- Completion Status (Completed / Pending)

All tasks are stored in localStorage. This allows persistence of data even after refreshing the browser without requiring a backend database.

The system supports full CRUD operations:
- Create (Add task)

- Read (Display tasks dynamically)
- Update (Edit task via modal)
- Delete (Remove task)
- Complete (Toggle completion status)

2. Detailed Documentation of Coding Decisions

2.1 Tasks Page – Technical Decisions

Bootstrap 5 was used for layout and responsiveness using the grid system. The form layout uses row and column classes to maintain alignment across devices.

Vanilla JavaScript with querySelector() was used instead of getElementById() to meet assignment constraints and improve selector flexibility.

Event delegation was implemented on the table body. This allows one event listener to handle dynamically created buttons (Edit, Delete, Complete), improving performance.

Filtering and sorting use a pipeline approach:
1. Start with full task array
2. Apply status filter
3. Apply priority filter
4. Apply sorting (name or date)
5. Render final result

The Edit feature uses a Bootstrap modal. When the Edit button is clicked, the modal fields are populated with the selected task's data. After saving, the task object is updated and the table re-renders.

2.2 Latest Activity Section

The Latest Activity section logs important user actions such as adding, editing, completing, or deleting tasks. Each log entry includes a timestamp.

Only the 10 most recent activities are stored to prevent excessive data growth.

2.3 Personal (About) Page

The About page uses Bootstrap Cards for team presentation and a Carousel component to showcase project highlights. Accessibility features such as alt text and ARIA attributes were included.

2.4 API Integration Decision

The Home page integrates the Ergast/Jolpi API to fetch live Formula 1 race data. A fallback mechanism was implemented to improve reliability in case of API failure.

2.5 Accessibility Considerations

Accessibility improvements include:
- Alt text for all images
- Proper label and input associations
- ARIA attributes for navigation and buttons
- Improved color contrast for dark mode
- Responsive navigation for keyboard accessibility

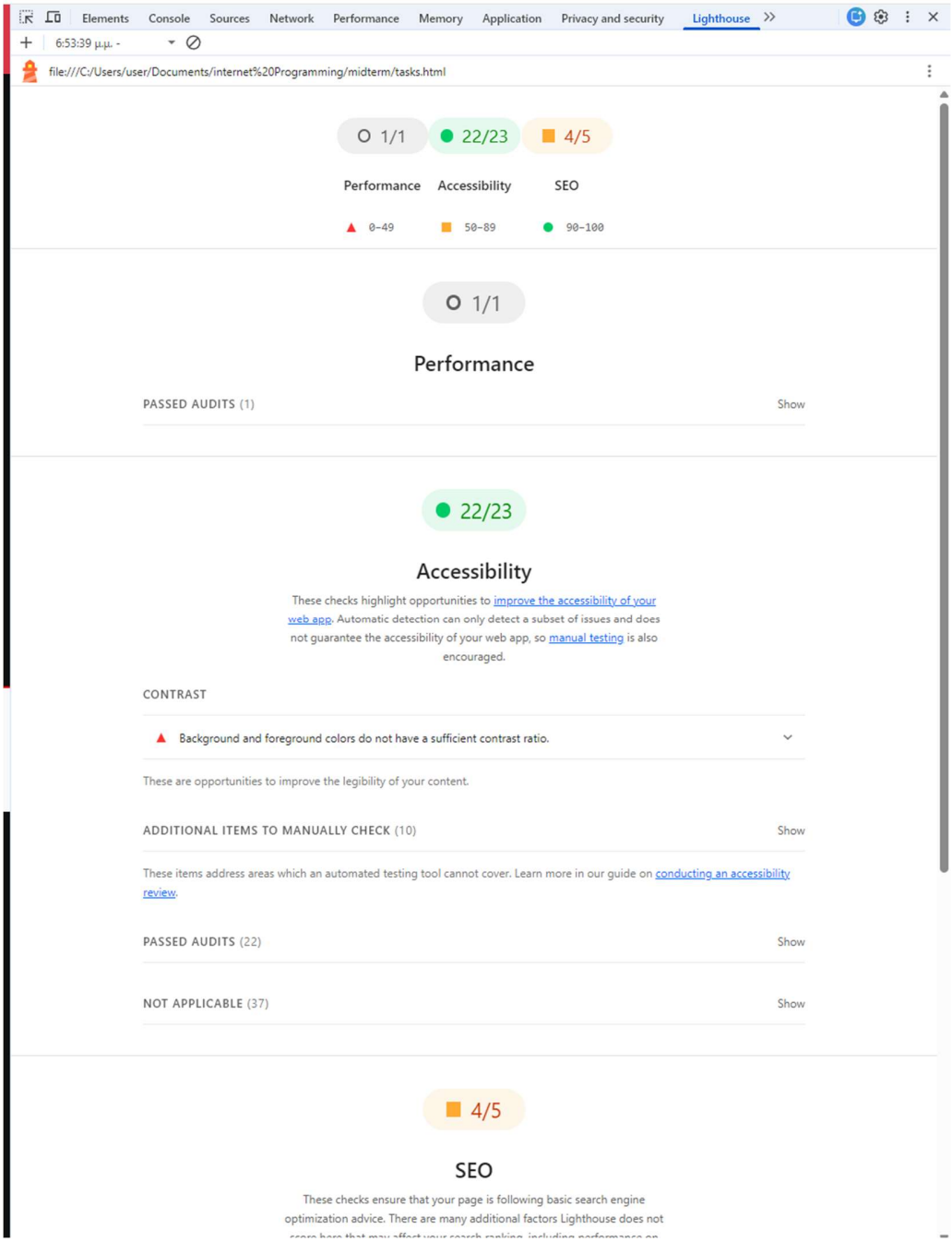
3. GitHub Repository

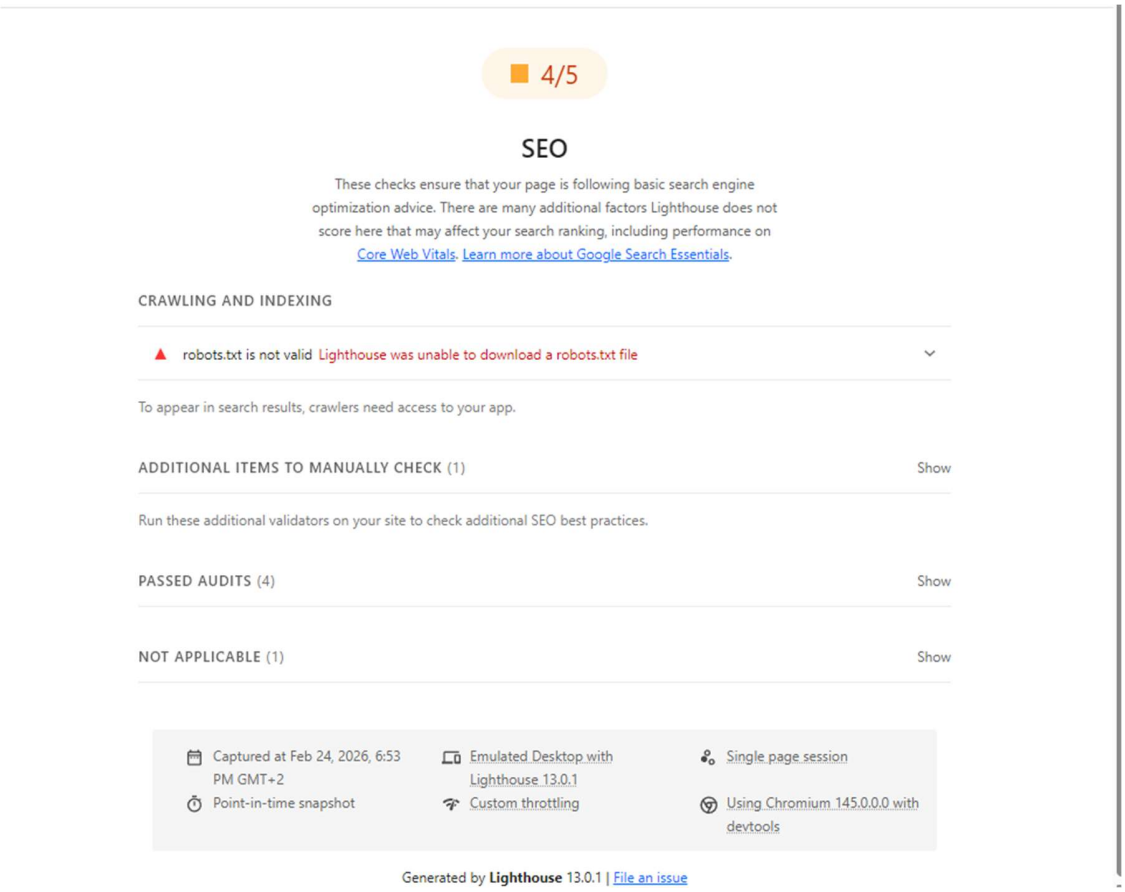
GitHub Link: https://github.com/Zervakos1/f1-formula-fans-site

4. Lighthouse Audit Results

The website was tested using Chrome Lighthouse to evaluate Accessibility, SEO, Performance, and Best Practices.

Accessibility: The site includes alt text for images, ARIA labels for navigation, proper form labels, and sufficient color contrast.
SEO: Meta tags, title tags, and semantic HTML elements were implemented.
Performance: Optimized layout and minimized JavaScript complexity.


Elements
Console
Sources
Network
Performance
Memory
Application
Privacy and security
Lighthouse
6:53:39 μμ. -
file:///C:/Users/user/Documents/internet%20Programming/midterm/tasks.html
1/1
22/23
4/5
Performance
Accessibility
SEO
0-49
50-89
90-100
1/1
Performance
PASSED AUDITS (1)
Show
22/23
Accessibility
These checks highlight opportunities to improve the accessibility of your web app. Automatic detection can only detect a subset of issues and does not guarantee the accessibility of your web app, so manual testing is also encouraged.
CONTRAST
Background and foreground colors do not have a sufficient contrast ratio.
These are opportunities to improve the legibility of your content.
ADDITIONAL ITEMS TO MANUALLY CHECK (10)
Show
These items address areas which an automated testing tool cannot cover. Learn more in our guide on conducting an accessibility review.
PASSED AUDITS (22)
Show
NOT APPLICABLE (37)
Show
4/5
SEO
These checks ensure that your page is following basic search engine optimization advice. There are many additional factors Lighthouse does not



5. Additional Components Implemented

1) FAQ Accordion (About Page)

Location: about.html (near the bottom, above the footer)
Component: Bootstrap Accordion
Purpose: The accordion provides an organized FAQ section where users can expand/collapse questions. This improves readability by reducing long blocks of text and supports accessibility because each question is a focusable button with proper ARIA attributes.

What it contains:

- "What is F1 Fan Hub?"
- "Where are tasks stored?" (localStorage explanation)
- "What does the Analytics page show?"

2) Toast Notifications (Tasks Page)

Location: tasks.html (toast container at the bottom-right of the page) and script.js (toast logic)
Component: Bootstrap Toast Notifications
Purpose: Toast notifications provide instant feedback after user actions on the Tasks page. Instead of only changing the table, the user gets a clear message confirming what happened. This improves user experience and makes the site feel like a real application.

When it appears:

- When a task is added
- When a task is marked completed / reopened
- When a task is edited
- When a task is deleted

3) Back-to-Top Floating Button

Location: Included in **about.html** near the end of the <body>, with styling in style.css and behavior in script.js.
Component: Floating "Back to Top" button
Purpose: This component improves navigation and accessibility by allowing users to quickly return to the top of a page, especially on pages with long content (About and Tasks). The button only appears after scrolling down and uses smooth scrolling when clicked.

Behavior:

- Hidden at the top of the page
- Appears after scrolling down ~250px
- Smooth scrolls to the top when clicked

6. AI Usage

AI tools were used as an assistant during this project to support learning and productivity. Specifically, AI was used to help brainstorm design ideas, suggest improvements for HTML/CSS layout, and provide guidance on JavaScript logic (such as task filtering/sorting and form validation). All code was reviewed, adapted, and integrated by me, and I tested the website functionality, fixed errors, and ensured the final implementation matched the assignment requirements. The final project reflects my own understanding and decisions, and I am able to explain the structure and code in detail.

7. Reflections

During the development of this project, several challenges were faced. One major challenge was ensuring that stored tasks remained compatible after modifying the task structure. This was solved by resetting or migrating localStorage data.

Another challenge was ensuring consistent dark mode styling across all pages. This required careful CSS overrides and testing.

API reliability was also a challenge, as some endpoints failed. A fallback mechanism was implemented to ensure data could still be retrieved.

Overall, this project improved my understanding of client-side frameworks, DOM manipulation, data persistence, accessibility standards, and responsive design.